CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: RevnU: A Sales Management System for Restaurants

Prepared By: Bigno, Richemmae Valle

Version: 3.2

Date: 05/20/2026

Status: Complete

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	02/18/2026	Richemmae Bigno	Initial draft	Draft
1.1	02/23/2026	Richemmae Bigno	Added changes to scope, added ERD and component diagram	Draft
1.2	03/03/2026	Richemmae Bigno	Finalized single-tenant logic; updated ERD; updated MoSCoW and User Journeys	Draft
2.0	04/16/2026	Richemmae Bigno	Updated Project Description	Draft
2.1	05/08/2026	Richemmae Bigno	Refactored naming to Entities	Draft
3.0	05/20/2026	Richemmae Bigno	Refactored user model: merged Owner into Restaurant profile; added analytics, admin account suspension/reactivation; confirmed no third-party payment integration	Draft
3.1	05/20/2026	Richemmae Bigno	Added predefined + custom category management for Sales and Expenses; updated categories table schema, seeded defaults, added CRUD endpoints and acceptance criteria	Draft
3.2	05/20/2026	Richemmae Bigno	Renamed non-Admin role from RESTAURANT to RESTAURATEUR throughout all sections	Complete

TABLE OF CONTENTS

EXECUTIVE SUMMARY

1.1 Project Overview

RevnU is a restaurant management system designed to digitize and streamline financial and operational tracking for restaurants. The system enables registered restaurants to record sales, expenses, manage staff payroll, and automatically compute profits. Owner contact information is optionally stored as part of the restaurant profile — no separate user type is required.

The system is composed of a Spring Boot backend API, a React web application, and an Android mobile application, all integrated through RESTful APIs to provide consistent and secure data access across platforms.

1.2 Objectives

1. Develop a functional restaurant management system focused on financial tracking and daily operations.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android Kotlin (mobile).
3. Design secure and maintainable RESTful APIs for cross-platform communication.
4. Provide a simple, intuitive interface suitable for non-technical users.
5. Deploy the system to a cloud-based environment for production and real usage.
6. Enforce security through JWT authentication, BCrypt password hashing, and Role-Based Access Control (RBAC) with Admin and Restaurateur roles.
7. Integrate features including a public holiday API, Google OAuth login, file uploads, SMTP email notifications, analytics, and admin account management.

1.3 Scope

Included Features:

- User authentication via Google OAuth 2.0 and JWT-based session management
- User registration and login functionality
- User roles (Admin / Restaurant) enforced at API and UI levels
- Admin ability to suspend or reactivate Restaurateur accounts
- Daily sales recording with categorized entries and variable pricing
- Expense tracking with automated profit computation
- Staff directory and payroll management (payroll treated as a form of expense)
- Predefined and custom category management for Sales and Expenses (system defaults + restaurant-defined)
- End-of-day closing to finalize and lock daily records
- End-of-day summary report delivered via SMTP email

- Analytics dashboard (revenue trends, expense breakdown, net profit)
- Integration with Philippine Public Holiday API
- File upload functionality (images/PDFs) linked to expense or related records
- React web application for full system management
- Android mobile application for sales and expense entry
- Spring Boot REST API using layered architecture
- PostgreSQL relational database

Excluded Features:

- Customer ordering or Point-of-Sale (POS) functionality
- Third-party payment processing or online payment integration
- Advanced inventory automation
- Data export to PDF or CSV formats
- Push notifications
- Offline functionality
- Multi-tenant staff registration or employee-level sub-accounts

1.0 INTRODUCTION

1.1 Purpose

This document defines the system design for RevnU, a restaurant management application. Restaurants register and log into the system to digitize financial tracking. It serves as a reference for system functionality, architecture, and the scope of Admin and Restaurateur operations.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: RevnU
Domain: Restaurant Management
Primary Users:

1. System Administrators (Admin)
2. Restaurateur

Problem Statement: Many small restaurants and food establishments rely on manual methods such as notebooks or spreadsheets to record daily sales, expenses, and employee salaries. These processes are prone to human error, lack data consistency, and make it difficult to generate accurate financial summaries or monitor business performance effectively.

Solution: RevnU provides a centralized digital platform that automates financial calculations, enforces structured data entry, and generates real-time daily and monthly summaries. The system integrates authentication, role-based authorization, file handling, public holiday API consumption, analytics, and automated email notifications.

2.2 Core User Journeys

Journey 1: Secure Login and Role Identification

1. User registers or logs in using email/password or Google OAuth.
2. System validates credentials and issues a JWT token.
3. Backend identifies the role as Admin or Restaurateur.
4. Dashboard loads with permissions restricted to the identified role.

Journey 2: Sales Recording

1. Restaurateur logs in and selects the Sales module from the sidebar.
2. Restaurateur clicks '+ Add Sales' to open the Add Sale Entry form.
3. Restaurateur enters an amount and selects a pre-defined category.
4. System validates input and links the record to the Restaurant's unique restaurant_id.
5. New entry appears at top of the list; Total Sales dashboard widget recalculates immediately.

Journey 3: Expense Recording with File Upload

1. Restaurateur navigates to the Expenses module.

2. Restaurateur clicks '+ Add Expense' and enters amount, category, and optional notes.
3. Restaurateur uploads a digital receipt (image or PDF) using the mobile camera or web file picker.
4. System processes the file, creates a Files record, and links file_id to the new Expense record.
5. Net Profit widget on the dashboard updates automatically.

Journey 4: Staff Management and Payroll

1. Restaurateur navigates to the Staff Management module.
2. Restaurateur creates a staff profile (Name, Position, Salary Rate) — staff are internal records, not system users.
3. Restaurateur records a payout via the Payout interface.
4. System logs the payout as a labor expense and updates net profit in real-time.

Journey 5: Restaurant Profile and Configuration Setup

1. After registration, Restaurateur navigates to the Restaurant Profile section.
2. Restaurateur enters details: Restaurant Name, Physical Address, Operating Hours.
3. Restaurateur optionally adds Owner Details (name, contact email, phone) as profile fields.
4. Restaurateur uploads a Business Logo; system stores the file and links the URL to the restaurants table.
5. System persists settings isolated to the Restaurateur's restaurant_id.

Journey 6: End-of-Day (EOD) Closing

1. Restaurateur reviews aggregated daily sales, expenses, and staff payouts on the dashboard.
2. Restaurateur initiates the reporting process via Soft Closing.
3. System marks the day's records as 'finalized' to prevent accidental modifications.
4. System calculates final net profit and sends an SMTP email summary to the registered email.

Journey 7: Holiday Context from Public API

1. Restaurateur opens the web or mobile dashboard.
2. System queries the Philippine National Holiday API for the current date.
3. If the date is a holiday, the dashboard displays a Holiday Status Badge next to the sales charts.
4. Restaurateur uses this context to evaluate why sales are higher or lower than usual.

Journey 8: Analytics

1. Restaurateur navigates to the Analytics module.
2. System renders charts and summaries: revenue trends (daily/weekly/monthly), expense breakdown by category, net profit over time.
3. Restaurateur uses this data to make informed business decisions.

Journey 9: Admin Account Management

1. Admin logs in and navigates to the User Management dashboard.
2. Admin can view all registered Restaurateur accounts with their status (Active / Suspended).
3. Admin clicks 'Suspend' to deactivate a Restaurateur account — the account can no longer log in.
4. Admin clicks 'Reactivate' to restore access to a previously suspended account.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration and login (Email/Password + Google OAuth 2.0)
2. JWT authentication with BCrypt password hashing
3. /me endpoint for authenticated user identification
4. RBAC: ADMIN and RESTAURATEUR roles with endpoint-level restrictions
5. Admin ability to suspend and reactivate Restaurateur accounts
6. Full CRUD: Sales, Expenses, and Staff
7. Payroll management (salary payouts recorded as labor expenses)
8. File upload and retrieval for restaurant logos and expense documentation
9. Profit computation (sales – expenses – payroll)
10. End-of-Day transaction closing with record locking
11. SMTP email notifications for account and EOD events
12. Public Philippine National Holiday API integration
13. React Web Application
14. Android Mobile Application (limited functionality)

SHOULD HAVE

15. Analytics dashboard (revenue trends, expense breakdown, net profit charts)
16. Monthly financial summaries
17. Search and filtering for sales and expenses

18. Structured validation and user-friendly error handling

COULD HAVE

19. Basic inventory tracking
20. Export functionality (PDF/CSV)

WON'T HAVE

21. POS and third-party payment processing
22. Push notifications
23. Staff-level registration or individual employee login credentials
24. Offline functionality

2.4 Detailed Feature Specifications

Feature: User Authentication

- Screens: Registration, Login, Forgot Password
- Fields: Email, Password, Confirm Password
- Validation: Email format, password strength, uniqueness
- API Endpoints: POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me
- Security: BCrypt password hashing, Google OAuth token validation, JWT issuance and validation, Secure session management

Feature: Role-Based Access Control (RBAC)

- Screens: Dashboard (Conditional view based on Admin or Restaurateur)
- Functions: ADMIN — Platform-level oversight and user management; RESTAURATEUR — Exclusive access to their restaurant's logs, configuration, analytics, and reporting
- API Endpoints: GET /api/admin/users, PUT /api/admin/users/{userId}/status, PUT /api/admin/users/{userId}/role
- Security: Spring Security @PreAuthorize with role permissions

Feature: Admin Account Management

- Screens: User Management Dashboard
- Functions: View all Restaurateur accounts; suspend an account (blocks login); reactivate a suspended account

- API Endpoints: PUT /api/admin/users/{userId}/suspend, PUT /api/admin/users/{userId}/reactivate
- Logic: Suspended accounts receive a 403 Forbidden response on login attempt

Feature: Restaurant Profile and Configuration

- Screens: Guided Configuration
- Display: Define restaurant name, physical address, operating hours
- API Endpoints: POST /api/settings/business,
- Validation: Progress checklist to guide the Restaurant through full profile completion

Feature: Sales Management

- Screens: Sales Entry, Sales Summary
- Display: Add, view, and update daily sales records
- Category Selection: Exactly one category must be selected per sale record; dropdown shows predefined categories first
- API Endpoints: POST /api/sales, GET /api/sales?date={date}, PUT /api/sales/{id}, DELETE /api/sales/{id}
- Validation: Mandatory amount and category_id fields; daily record locking after EOD

Feature: Expense Management and File Upload

- Screens: Expense Entry, Receipt Viewer
- Functions: Record operational costs with optional camera-based receipt photo uploads
- Category Selection: Exactly one category must be selected per expense record; dropdown shows predefined categories first
- API Endpoints: POST /api/expenses, GET /api/expenses, POST /api/expenses/{id}/upload
- Security: Multipart file handling; receipt images linked to database records and stored in cloud/server storage

Feature: Staff & Salary Management

- Screens: Staff Directory, Payout History
- Data: Internal records for employees used for labor cost tracking — staff are not system users

- Data Collected: Staff Name, Position, Salary Rate
- API: POST /api/staff, GET /api/staff, PUT /api/staff/{id}, DELETE /api/staff/{id}, POST /api/salaries
- Logic: Salary payouts are automatically subtracted from gross revenue to compute net profit

Feature: End-of-Day (EOD) Reporting

- Screens: Daily Summary Dashboard
- Functions: Soft Closing and Status Updates to finalize daily reports and compute net profit
- API Endpoints: POST /day/close, GET /day/summary/{date}
- Integration: Triggers an automated SMTP email containing the daily financial report to the Restaurant

Feature: Analytics

- Screens: Analytics Dashboard
- Functions: Visualize revenue trends (daily/weekly/monthly), expense breakdown by category, net profit over time
- API Endpoints: GET /api/analytics/summary, GET /api/analytics/revenue, GET /api/analytics/expenses
- Display: Line charts, bar charts, and summary cards

Feature: Public Holiday API Integration

- Screens: Dashboard, Sales Summary
- Functions: Consume public Philippine holiday data to provide context for sales performance
- API Endpoints: GET /api/external/holidays
- Display: Visual holiday badge displayed alongside financial metrics to explain sales trends

Feature: Automated Email Notifications (SMTP)

- Trigger: Account creation and End-of-Day closing
- Functions: Send automated Welcome emails and digital daily summary receipts
- Security: Integration with an SMTP server to ensure deliverability

2.5 Acceptance Criteria

AC-1: Successful User Registration and OAuth Login

- Given I am a new user,
- when I enter a valid email and a strong password, or use 'Sign-In with Google',
- and the system successfully validates the credentials or Google token,
- then my account should be created and saved in the database with a hashed password (BCrypt).

AC-2: Daily Sales Entry and Data Linking

- Given I am logged in as a Restaurateur with a valid JWT session,
- when I record sales for the day with a specific category and amount,
- then the system should accurately update daily totals,
- and the record must be linked to my specific restaurant_id to ensure data is isolated from other restaurants.

AC-3: Role-Based Access Control (RBAC)

- Given I am logged in with a 'Restaurateur' role,
- when I attempt to access the Admin user management dashboard,
- then the system should deny access with a '403 Forbidden' error at both the UI and API levels.
- Given I am logged in with an Admin role,
- when I attempt to record daily sales or initiate an EOD report,
- then the system should deny access with a '403 Forbidden' error.

AC-4: Expense Tracking with File Upload

- Given I have a physical receipt for a business expense,
- when I create an expense entry and upload an image or PDF,
- then the file should be stored on the server and linked to that specific expense record,
- and I should be able to view the file later.

AC-5: Internal Staff Management and Payroll

- Given I am logged in as a Restaurateur,
- when I create a staff record and log a salary payout,
- then the system should store the payout as a labor expense linked to the restaurant,
- and the payout amount must be subtracted from the gross revenue during profit computation.

AC-6: End-of-Day Closing and Automated Email

- Given all sales and expenses for the day are recorded,
- when the Restaurant initiates End-of-Day reporting,
- then the system should perform a Soft Closing (marking records as finalized) and automatically send a summary notification email (SMTP) to the restaurant's registered email.

AC-7: Admin Account Suspension and Reactivation

- Given I am logged in as Admin,
- when I suspend a Restaurateur account,
- then that account should receive a 403 Forbidden response on any subsequent login attempt.
- When I reactivate the account,
- then that Restaurateur should be able to log in and access their dashboard normally.

AC-8: Analytics Dashboard

- Given I am logged in as a Restaurateur,
- when I navigate to the Analytics module,
- then the system should display revenue trends, sale breakdowns by category, and net profit charts that accurately reflect my recorded data.

AC-9: Holiday Badge Display

- Given I am viewing the Sales Management or Dashboard screen,
- when the page loads on a Philippine public holiday,
- then the system should fetch real-time holiday data from the public API and display a holiday status badge alongside the day's financial metrics.

AC-10: Secure Data Handling (DTO Implementation)

- Given the system processes a request for user or financial data,
- when the API returns a response,
- then the system must utilize Data Transfer Objects (DTOs) to ensure sensitive information such as password hashes is never exposed.

AC-11: Single Category per Record

- Given I am recording a sale or expense,
- when I submit the form, then exactly one category must be selected — the system should reject the submission with a validation error if no category is chosen.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds for CRUD operations
- End-of-Day (EOD) locking and summary generation ≤ 3 seconds
- Support at least 50 concurrent users
- Database queries ≤ 500ms
- Caching for External API data to maintain UI fluidity
- File uploads (receipts) processed within reasonable time limits

3.2 Security Requirements

- HTTPS encryption for all client-server communications
- JWT-based authentication
- Password hashing using BCrypt
- Role-Based Access Control (RBAC) for Admin and Restaurateur roles
- Account suspension enforced at the authentication layer
- Input validation and SQL injection prevention
- File upload type and size restrictions
- Mandatory use of DTOs to prevent sensitive data exposure

3.3 Compatibility Requirements

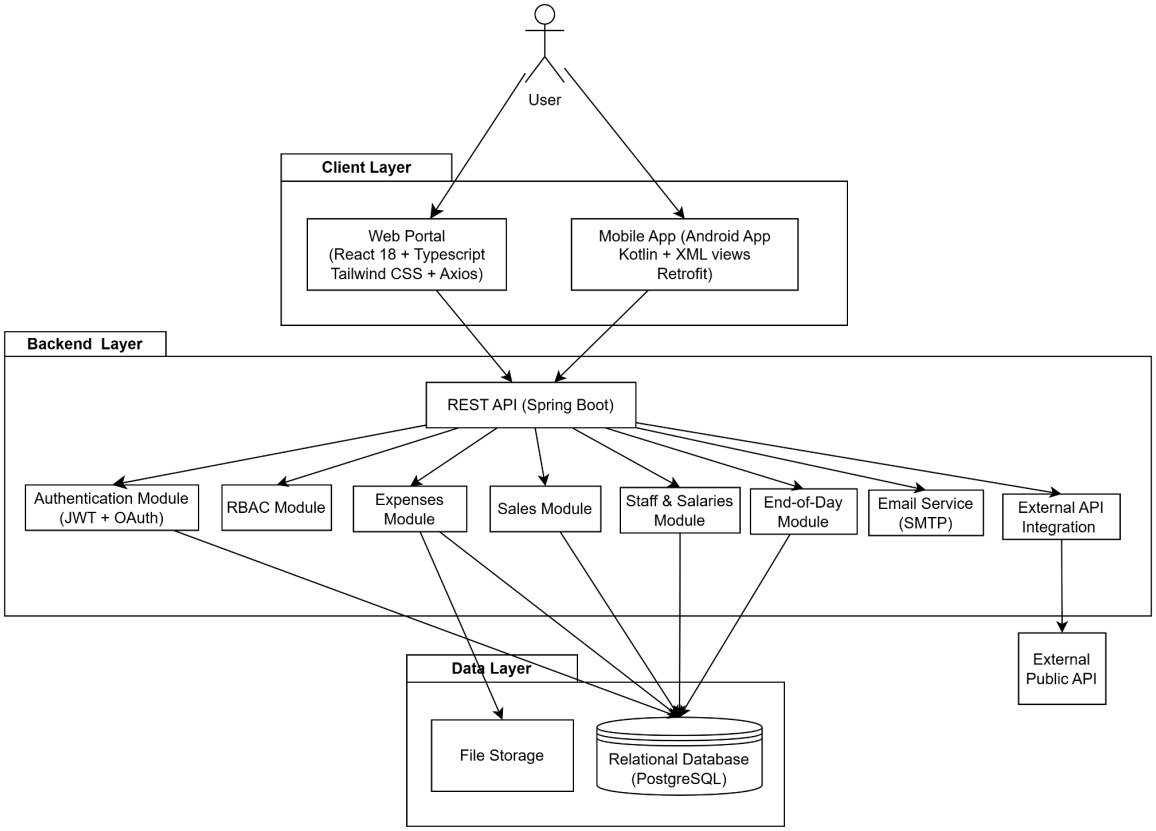
- Web: Chrome, Firefox, Edge (latest versions)
- Android: API Level 24+
- Screen sizes: Mobile, tablet, desktop
- OS: Windows, macOS, Linux

3.4 Usability Requirements

- Minimal and intuitive interface
- Daily tasks completable within minutes
- Clear error and validation messages
- Consistent navigation across all screens

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



4.2 Technology Stack

Layer	Technology
Backend	Java 17, Spring Boot 4.x, Spring Security (JWT + OAuth 2.0), Spring Data JPA
Database	PostgreSQL 14+ (Relational Database)
File Storage	Local Server Storage / Cloud Storage (receipt images and logos)
Web Frontend	React 18, TypeScript, Tailwind CSS, Axios
Mobile Frontend	Android Kotlin, XML views, Retrofit
External Integrations	Google OAuth 2.0, SMTP Server (Email), Philippine Public Holiday API
Build Tools	Maven (Backend), npm (Web), Gradle (Android)
Deployment	Render (Backend), Vercel (Web), APK Distribution (Mobile)

5.0 API CONTRACT AND COMMUNICATION

5.1 API Standards

Property	Value
Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	{ "success": boolean, "data": object|null, "error": { "code": string, "message": string, "details": object|null }, "timestamp": string }

5.2 Endpoint Specifications

User Registration

Property	Details
Description	Register a new Restaurateur account
API URL	/auth/register
HTTP Method	POST
Authentication	None
Request Payload	{ "email": <email>, "password": <password>, "restaurantName": <string> }
Response	{ success, data: { restaurant: { email, restaurantName, role }, accessToken, refreshToken }, error, timestamp }
Notes	Owner details (ownerName, ownerEmail, ownerPhone) can be added later via the Restaurant Profile settings

User Login

Property	Details
Description	Authenticate a user and obtain JWT tokens
API URL	/auth/login
HTTP Method	POST
Authentication	None
Request Payload	{ "email": <email>, "password": <password> }

Response	{ success, data: { user: { email, role }, accessToken, refreshToken }, error, timestamp }
Notes	Returns 403 Forbidden if the account has been suspended by Admin

Google OAuth Authentication

Property	Details
Description	Authenticate or register users using Google OAuth 2.0 tokens
API URL	/auth/google
HTTP Method	POST
Authentication	None
Request Payload	{ "idToken": <google_id_token> }
Response	{ success, data: { user: { email, role }, accessToken }, error, timestamp }

Sales Entry

Property	Details
Description	Records a new sales transaction
API URL	/api/sales
HTTP Method	POST
Authentication	Bearer token (JWT) — Restaurateur role only
Request Payload	{ "amount": <double>, "category": <string>, "note": <string> }
Response	{ success, data: { saleId: <long>, status: "recorded" }, error, timestamp }

End-of-Day Reporting

Property	Details
Description	Finalizes the daily ledger, locks records, and triggers SMTP report
API URL	/day/close
HTTP Method	POST
Authentication	Bearer token (JWT) — Restaurateur role only
Request Payload	{ "date": <yyyy-mm-dd> }

Response	{ success, data: { summary: { totalSales, totalExpenses, totalPayroll, netProfit }, reportSent: boolean }, error, timestamp }

Admin: Suspend Account

Property	Details
Description	Suspends a Restaurateur account; blocks subsequent logins
API URL	/api/admin/users/{userId}/suspend
HTTP Method	PUT
Authentication	Bearer token (JWT) — Admin role only
Request Payload	None
Response	{ success, data: { userId, status: "SUSPENDED" }, error, timestamp }

Admin: Reactivate Account

Property	Details
Description	Reactivates a previously suspended Restaurateur account
API URL	/api/admin/users/{userId}/reactivate
HTTP Method	PUT
Authentication	Bearer token (JWT) — Admin role only
Request Payload	None
Response	{ success, data: { userId, status: "ACTIVE" }, error, timestamp }

Analytics Summary

Property	Details
Description	Returns aggregated financial analytics for the Restaurant
API URL	/api/analytics/summary
HTTP Method	GET
Authentication	Bearer token (JWT) — Restaurateur role only
Query Params	?from=<date>&to=<date>&granularity=daily|weekly|monthly
Response	{ success, data: { revenue: [...], expenses: [...], netProfit: [...] }, error, timestamp }

Get Categories

Property	Details
Description	Fetch all categories (predefined + restaurant's own custom) filtered by type
API URL	/api/categories
HTTP Method	GET
Authentication	Bearer token (JWT) — Restaurateur role only
Query Params	?type=SALES or ?type=EXPENSE (optional; omit for all)
Response	{ success, data: [{ id, name, type, isDefault, restaurantId }, ...], error, timestamp }
Notes	Predefined categories have restaurantId = null and isDefault = true; shown first in the list

5.3 Error Handling

HTTP Status Codes

- 200 OK — Successful request
- 201 Created — Resource created
- 400 Bad Request — Invalid input
- 401 Unauthorized — Authentication required/failed
- 403 Forbidden — Insufficient permissions (or account suspended)
- 404 Not Found — Resource doesn't exist
- 409 Conflict — Duplicate resource
- 500 Internal Server Error — Server error

Error Code Examples

Example 1	json
	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},

	"timestamp": "2024-01-28T10:30:00Z" }

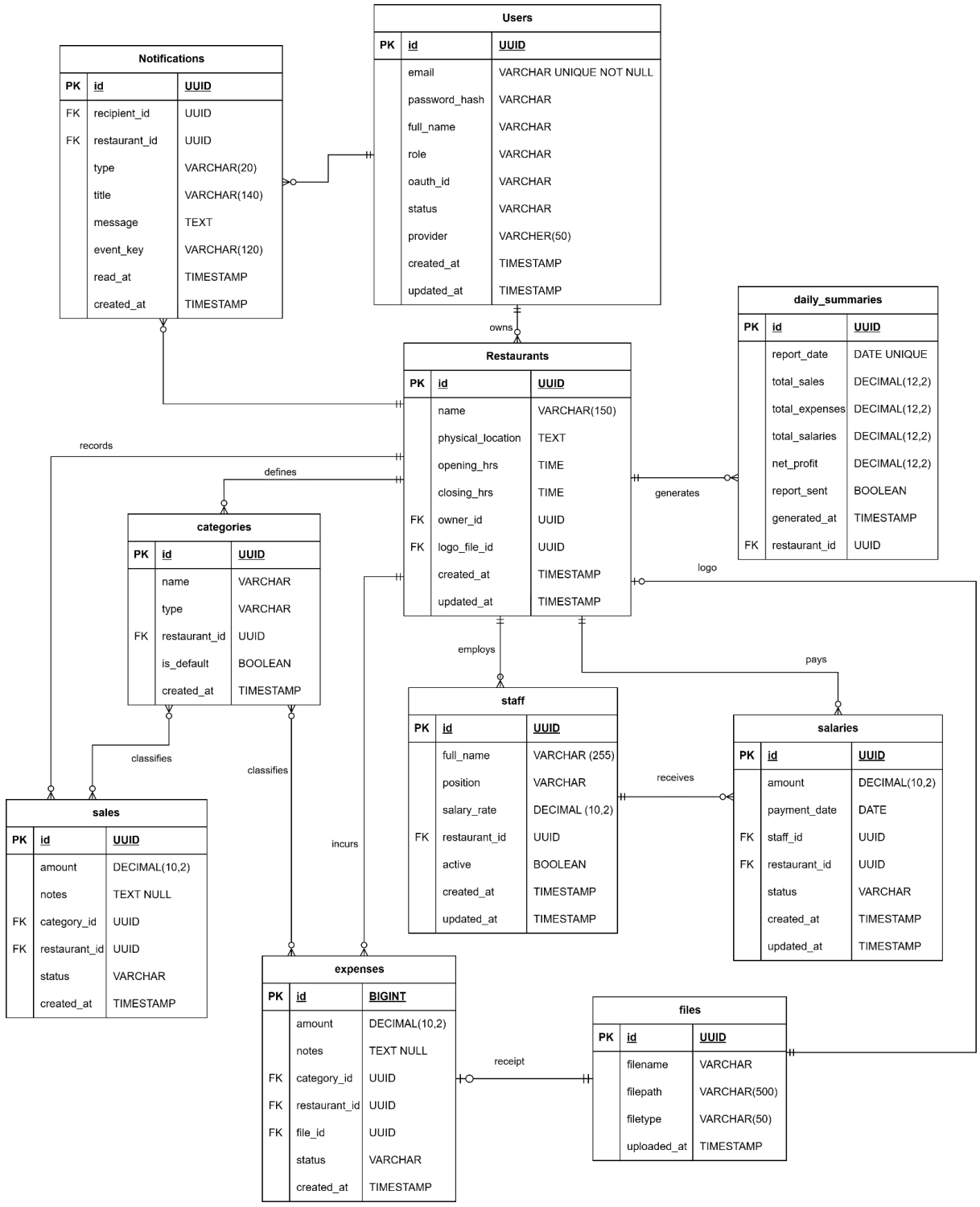
Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- AUTH-004: Account suspended
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- One-to-Many: users → restaurants — a user can own one restaurant; the restaurant stores owner_id as a foreign key
- One-to-Many: restaurants → staff — internal employee records scoped to a restaurant, not system users
- One-to-Many: restaurants → sales — all revenue records are scoped to a restaurant_id
- One-to-Many: restaurants → expenses — all cost records are scoped to a restaurant_id
- One-to-Many: restaurants → salaries — salary payouts are scoped to a restaurant_id for reporting
- One-to-Many: restaurants → daily_summaries — one EOD summary record per day per restaurant
- One-to-Many: restaurants → notifications — notifications are scoped to a restaurant context
- One-to-Many: staff → salaries — an employee can have multiple payout records over time
- One-to-Many: categories → sales — each sale is classified under one category
- One-to-Many: categories → expenses — each expense is classified under one category
- One-to-Many: users → notifications — a user receives multiple notifications as recipient_id
- Many-to-One: expenses → files — an expense optionally links to one receipt file upload
- Many-to-One: restaurants → files — a restaurant optionally links to one logo file upload

Key Tables:

1. **users:** Stores account credentials, BCrypt hashed passwords, and Google OAuth links for Admins and Tenants.
2. **restaurants:** Global configuration including name, logo, location, and owner details.
3. **categories:** Predefined labels for Sales and Expenses
4. **staff:** Internal records of restaurant employees (non-users) used for payroll tracking.
5. **sales:** Revenue tracking records with status indicators for soft-closing.
6. **expenses:** Business cost records with links to digital receipt files.

7. **salaries:** Payout history for internal staff used in financial profit computations.
8. **files**: Storage metadata for uploaded business logos and receipt images.
9. **daily_summaries:** Historical record of daily financial performance and net profit.
10. **Notifications:** In-app alerts delivered to users for system events such as holiday advisories and end-of-day reports.

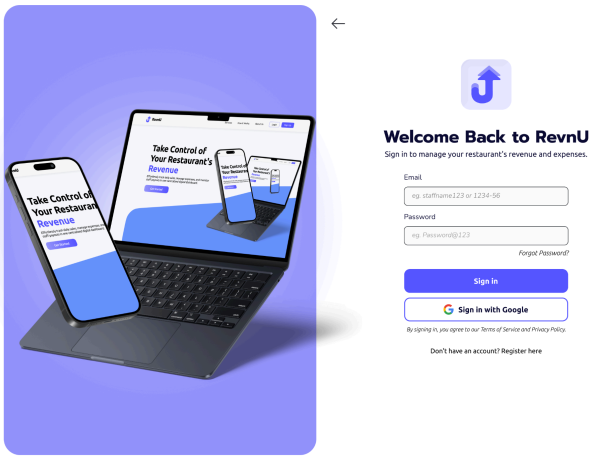
Table Structure Summary:

- **users:** id, fullname, email, password_hash, role, status, oauth_id, created_at
- **restaurants:** id, name, logo_file_id, physical_location, opening_hrs, closing_hrs, owner_id
- **categories:** id, name, type (SALES / EXPENSE), restaurant_id, is_default, created_at
- **staff:** id, fullname, position, salary_rate, restaurant_id
- **sales:** id, amount, category_id, notes, restaurant_id, status (OPEN / CLOSED), created_at
- **expenses:** id, amount, category_id, notes, restaurant_id, file_id, status (OPEN / CLOSED), created_at
- **salaries:** id, amount, payment_date, staff_id, restaurant_id, status (OPEN / CLOSED), created_at
- **files: i**d, filename, filepath, filetype, uploaded_at
- **daily_summaries:** id, report_date, total_sales, total_expenses, total_salaries, net_profit, report_sent, generated_at, restaurant_id
- **notifications:** id, recipient_id, restaurant_id, type, title, message, event_key, read_at, created_at

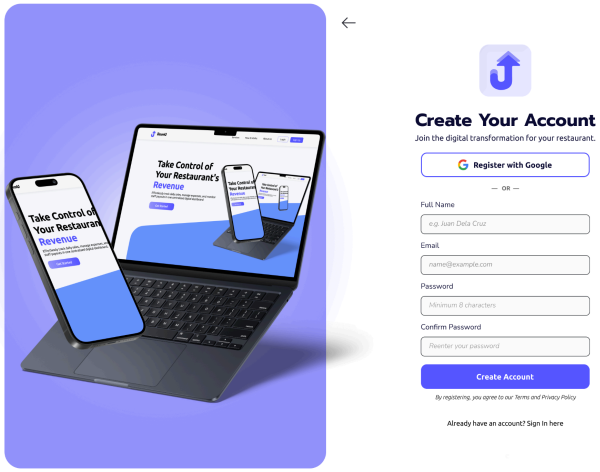
7.0 UI/UX DESIGN

7.1 Web Application Screens

Login

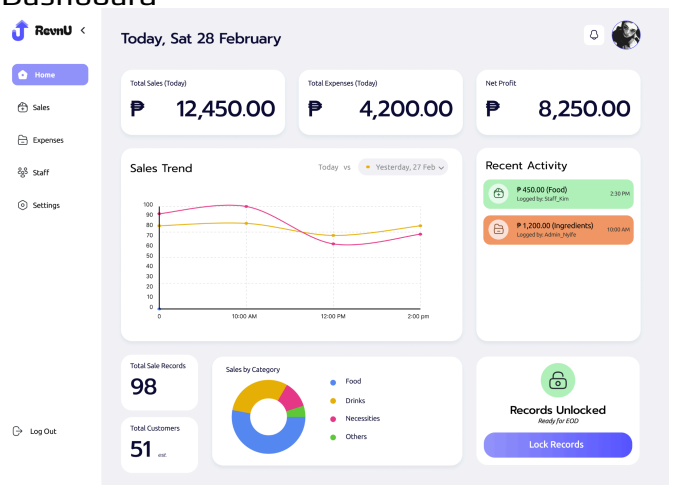


Register

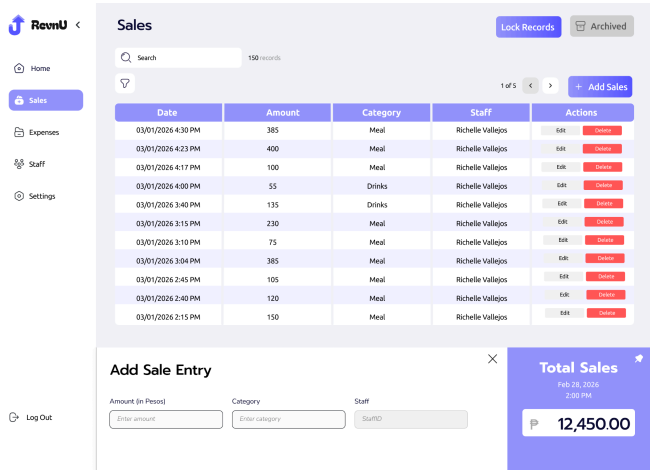


Setup

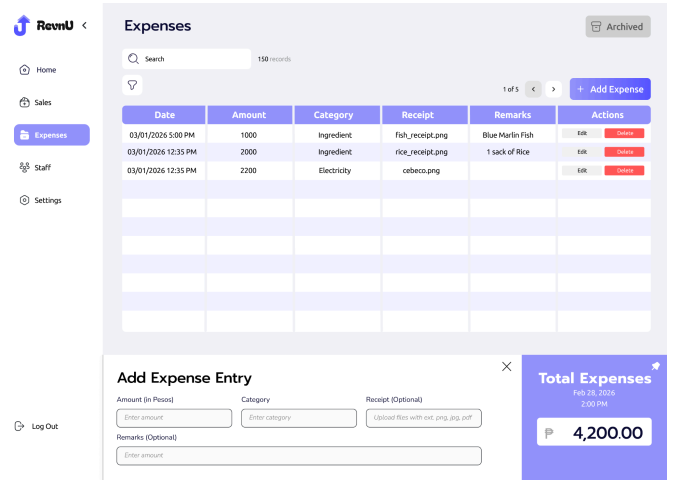
Dashboard



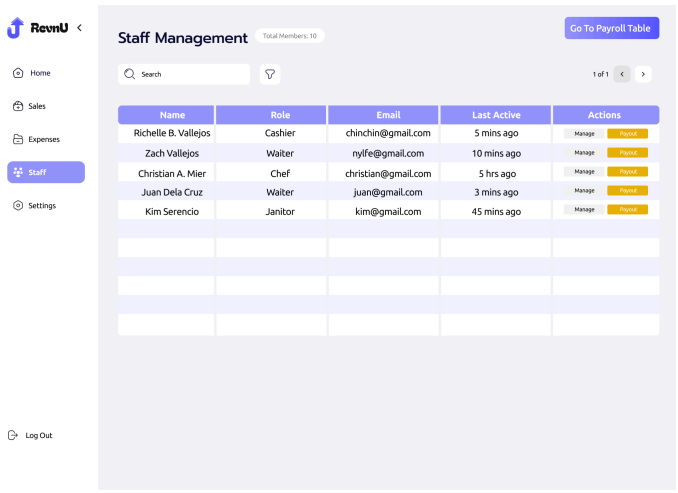
Sales Tracker



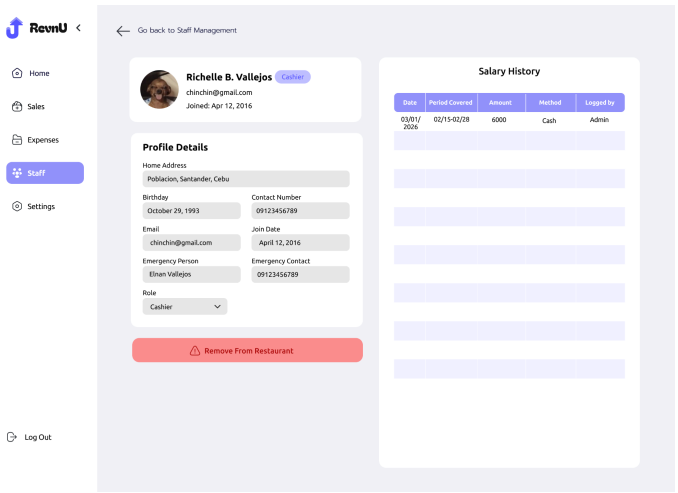
Expenses Tracker



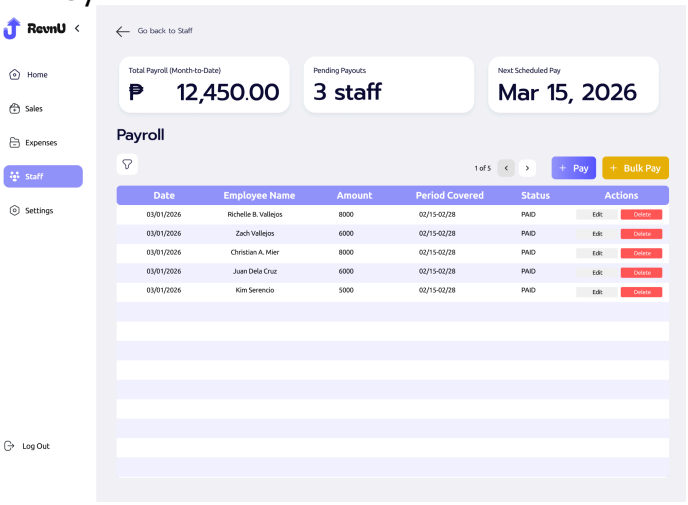
Staff Management



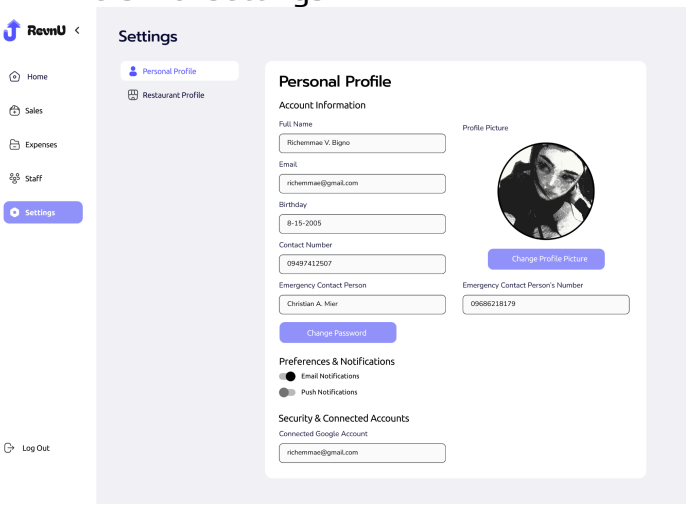
Staff Profile



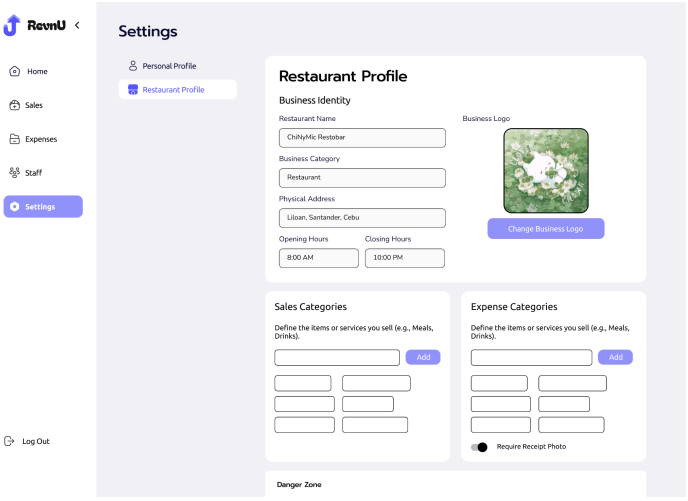
Payroll



Personal Settings



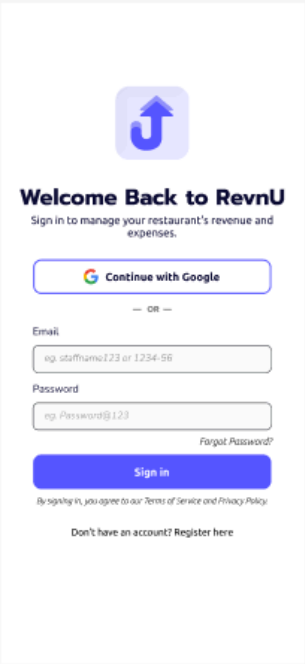
Restaurant Settings



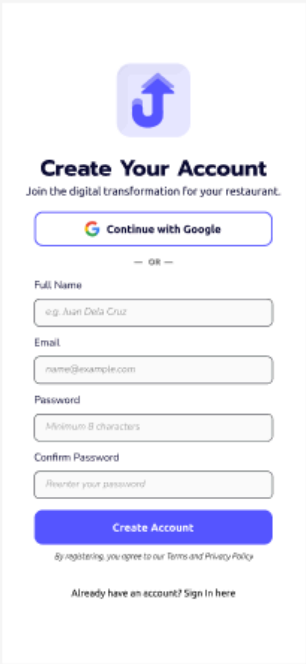
7.2 Mobile Application Screens

Splash

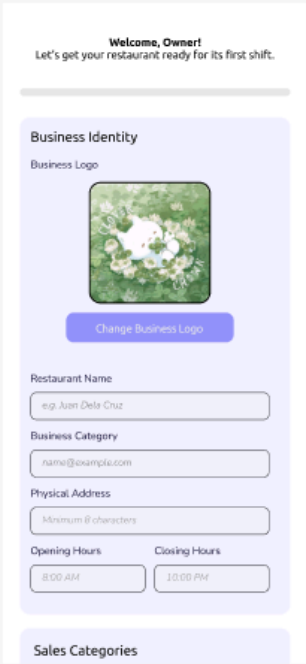
Login



Registration

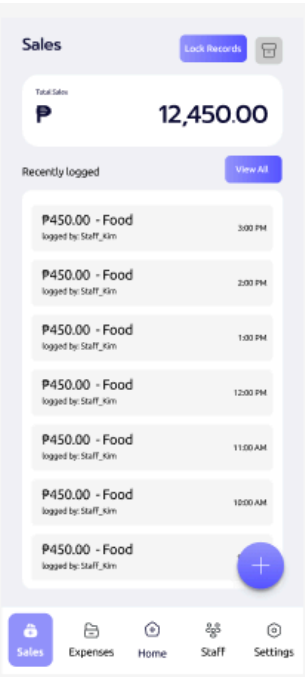


Onboarding

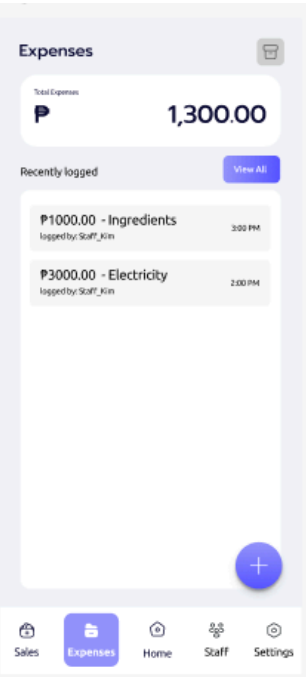


Dashboard

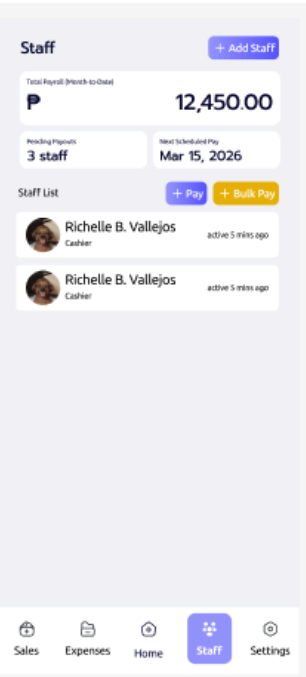
Sales



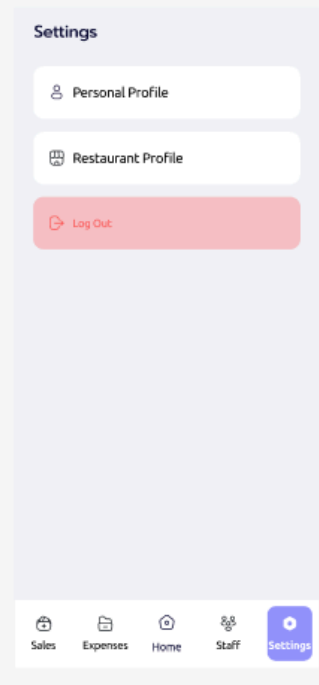
Expenses



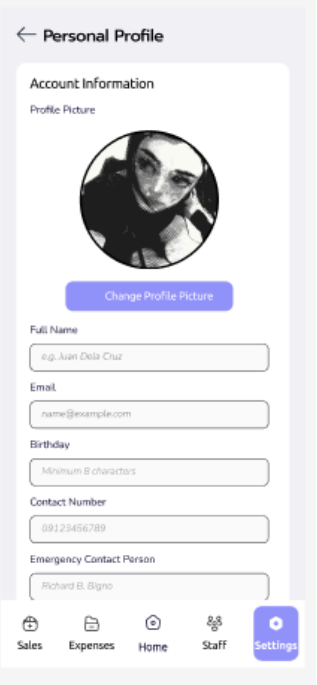
Staff List



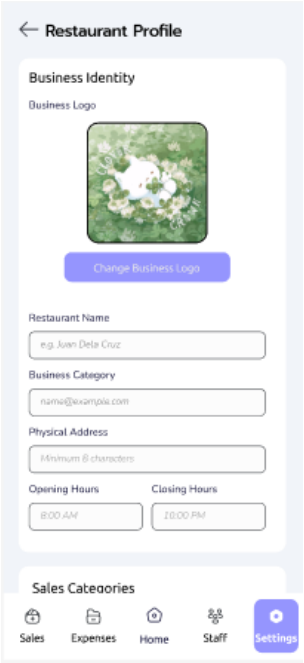
Settings



Personal Profile



Restaurant Profile



8.0 PLAN

8.1 Project Timeline

Phase 1: Planning and Design (Weeks 1–2)

Week 1: Requirements and Architecture

- Day 1–2: Project setup; define Admin and Restaurateur boundaries and data flow.
- Day 3–4: Complete FRS focusing on Sales, Expense, Staff, Analytics, and Account Management.
- Day 5–7: Finalize three-tier architecture: Spring Boot + React + Android.

Week 2: Detailed Design

- Day 1–2: API specification for Sales, Expenses, Staff, Analytics, EOD, and Admin endpoints.
- Day 3–4: Database design;
- Day 5–6: UI/UX wireframes for Mobile Sales Keypad, Web Analytics Dashboard, Admin User Management.
- Day 7: Implementation plan and ERD finalization.

Phase 2: Backend Development (Weeks 3–4)

Week 3: Foundation and Identity

- Day 1: Spring Boot setup with Security and JPA dependencies.
- Day 2: Database configuration and Entity mapping.
- Day 3: Google OAuth 2.0 Integration and RBAC for Admin and Restaurateur roles.
- Day 4: Restaurant onboarding logic and Profile creation
- Day 5: Account suspension/reactivation endpoints; /me endpoint implementation.

Week 4: Core Business Logic

- Day 1: Sales Logging API with Category filtering and restaurant isolation.
- Day 2: Expense Tracking API with Multipart image upload support.
- Day 3: Analytics API endpoints.
- Day 4: Staff CRUD and Salary Payout logic.
- Day 5: SMTP Email service.

Phase 3: Web Application (Weeks 5–6)

Week 5: Management Hub

- Day 1–2: React setup and Onboarding Dashboard.
- Day 3: Admin User Management screen (view/suspend/reactivate).
- Day 4: Sales and Expense Ledgers with filtering; Staff Directory.
- Day 5: Analytics Dashboard with charts (Revenue, Expenses, Net Profit, Date Filters).

Week 6: Advanced Features

- Day 1: EOD Soft Closing workflow and SMTP Daily Report generation.
- Day 2: Payroll payout interface.
- Day 3: Public Holiday API integration and Holiday Badge.
- Day 4: Responsive design polish.
- Day 5: End-to-end integration testing and bug fixes.

Phase 4: Mobile Application (Weeks 7–8)

Week 7: Android Sales Client

- Day 1: Android Studio setup and Retrofit service layer.
- Day 2: Google Sign-In integration.
- Day 3: Sales Keypad UI with category chips.
- Day 4: Expense Logger with camera intent for receipt capture.
- Day 5: Daily logs view for real-time revenue tracking.

Week 8: Experience and Polish

- Day 1: Staff Salary History view.
- Day 2: UI haptics and success feedback for rapid sales entry.
- Day 3–4: Real-world testing on physical Android devices.
- Day 5: APK generation and mobile technical documentation.

Phase 5: Integration and Deployment (Weeks 9–10)

Week 9: System Hardening

- Day 1: Integration testing: ensuring mobile entries sync with the Web Dashboard.
- Day 2–3: Security review — Restaurants cannot access Admin endpoints; suspended accounts blocked.

- Day 4: Performance testing for API response times under load.
- Day 5: Documentation updates and Final System Audit.

Week 10: Deployment

- Day 1: Backend deployment (Render) with PostgreSQL.
- Day 2: Web app deployment (Vercel) and domain setup.
- Day 3: Final APK distribution.
- Day 4: Project Submission.

8.2 Milestones

Milestone	Description
M1 (End Wk 2)	Design docs and wireframes for Admin/Restaurant flows complete
M2 (End Wk 4)	Backend API supports Auth, CRUD for Sales/Expenses, Analytics, and Account Suspension
M3 (End Wk 6)	Web portal handles Restaurant Setup, Analytics Dashboard, and Soft Closing reporting
M4 (End Wk 8)	Mobile app functional for fast sales entry and receipt capture
M5 (End Wk 10)	Full system live; SMTP reports sending daily to Restaurants

8.3 Risk Mitigation

- Start with the simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements
- Validate account suspension logic end-to-end before deployment